Carpeta de campo

Alumno: Nicolás Rosato

Docentes: Nahuel Acosta/Gabriel Abello

Proyecto: Empresa de Logística

Integrantes del proyecto: Xiomara Ureña, Santiago Pedraza, Santiago Goldfard, Benjamin Saragusti, Agustin Zowmir

Año y Curso : 5° 3° computación Turno: Mañana

1. Introducción y Fundamentación

El presente proyecto académico tiene como propósito el diseño, modelado e implementación integral de un sistema de software para una Empresa de Logística, estructurado de manera que emule con precisión las operaciones y flujos de trabajo del sector en el mundo real.

Objetivos del Proyecto

- Objetivo General:
  Recrear de forma sistémica el funcionamiento de una empresa de logística moderna, desarrollando una plataforma tecnológica capaz de soportar, automatizar y gestionar sus operaciones comerciales y de distribución principales.
- Objetivos Específicos:
  - Garantizar la trazabilidad y el control riguroso de las existencias mediante la vinculación precisa entre productos y paquetes.
  - Implementar una arquitectura de persistencia de datos segura, íntegra y eficiente a través de un sistema de gestión de bases de datos relacionales.
  - Desarrollar una interfaz de usuario intuitiva que incorpore estrictos mecanismos de validación y seguridad para mitigar errores en el flujo de ejecución.

A través de esta implementación, se busca consolidar los conocimientos de programación orientada a objetos, diseño de bases de datos y gestión de proyectos adquiridos a lo largo del ciclo lectivo.

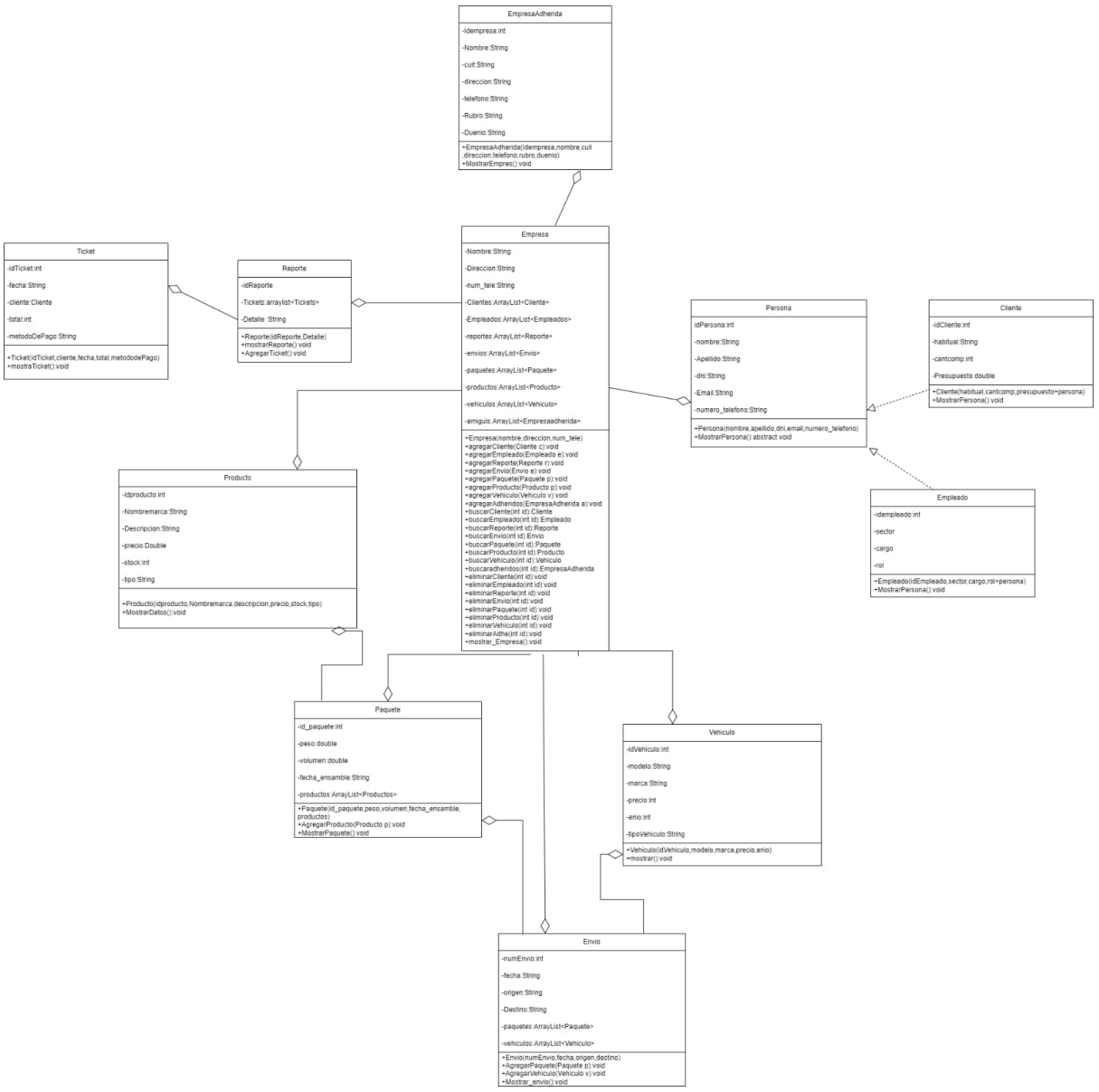
11/07

Se procedió a la estructuración, abstracción y organización lógica de la clase Empresas Adheridas. Durante esta jornada, se definieron detalladamente sus campos de conocimiento (atributos privados) y sus métodos correspondientes de acceso y mutación, asegurando la correcta encapsulación de los datos y la abstracción de la lógica de negocio asociada a dicha entidad.

15/07

Se realizó la transposición de la clase previamente diseñada al entorno de desarrollo integrado, codificándola de manera efectiva bajo el paradigma de Programación Orientada a Objetos en el lenguaje Java. En colaboración con el compañero Agustín Zowmir, se llevaron a cabo las revisiones técnicas y los ajustes finales sobre el diagrama de clases UML del sistema, optimizándolo para eliminar inconsistencias y asegurar una representación fiel y consolidada del modelo de datos hasta la fecha.

Tras la finalización del receso invernal, se asistió a la primera instancia de preentrega presencial. Los profesores Nahuel Acosta y Gabriel Tuinyla efectuaron una auditoría técnica sobre la totalidad de los entregables y artefactos producidos hasta el momento. El resultado de la evaluación general fue satisfactorio, detectándose únicamente desvíos menores. Siguiendo las directrices del cuerpo docente, se procedió a la eliminación de múltiples tablas del diagrama UML debido a problemas de redundancia y normalización de la base de datos. Asimismo, se actualizó la matriz de asignación de tareas y el cronograma de actividades planificadas para las siguientes etapas del proyecto. Posterior a la sesión, se ejecutó de forma inmediata la reestructuración y actualización del diagrama UML en conjunto con el compañero Agustín Zowmir.



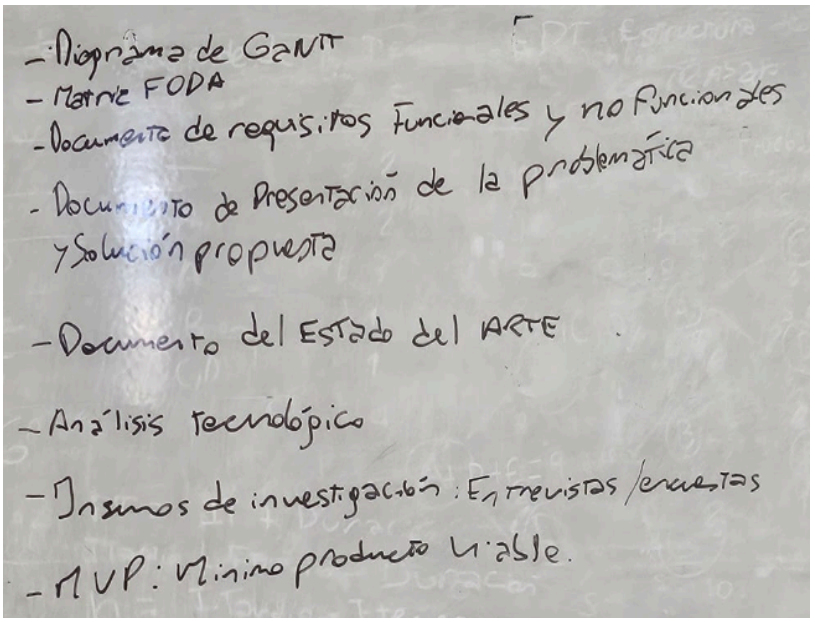
6/08

Durante el bloque horario correspondiente a la materia del profesor Nahuel Acosta, el equipo completo se reunió con el objetivo de planificar, diseñar y presupuestar la infraestructura de red local y perimetral para la empresa

de logística. En esta sesión de trabajo colaborativo, se realizó un relevamiento de mercado de los componentes tecnológicos requeridos, orientando la búsqueda a minimizar el gasto económico y priorizando soluciones de alta eficiencia a bajo costo bajo criterios de optimización presupuestaria.

Los componentes seleccionados para el listado final de la infraestructura fueron estaciones de trabajo completas con sus respectivos periféricos, un switch Gigabit Ethernet de 24 puertos para la capa de acceso, un router Cisco 2811 equipado con dos puertos FastEthernet para el enrutamiento perimetral, cables de red de parcheo de dos metros con soporte de transmisión de hasta 40 Gbps, cableado coaxial fino para segmentos específicos, terminales de telefonía IP, unidades de impresión láser para la gestión administrativa, un plotter de corte láser de 63 centímetros para el área de etiquetado y paquetería, y cámaras de seguridad domo con soporte de audio bidireccional de 360 grados y resolución de 2MP (marca Imou Ranger SE con sistema de alarma integrado). El análisis financiero consolidado arrojó un presupuesto final total de $695.841.416, el cual quedó registrado formalmente en una hoja de cálculo optimizada de Google Sheets.

El profesor Gabriel Tuinyla formalizó y entregó a todos los grupos de trabajo el listado de requerimientos técnicos, hitos de control y entregables obligatorios que debían ser subsanados y presentados para la siguiente instancia de preentrega programada para el día 14 de agosto.



Dado el listado de las cosas que tenemos que tener hechas para la siguiente pre entrega Benjamin se encargó de dividir todas las consignas entre todos los miembros del grupo. A mi se me asignó realizar la documentación del estado de arte junto a mi compañera Xiomara Ureña.

9/8

Este día con mi compañero Xiomara Ureña llevamos a cabo una investigación exhaustiva orientada al análisis de la competencia con el fin de identificar fortalezas, debilidades, ventajas y desventajas del sector logístico. El estudio se centró en las firmas Andreani, Correo Argentino y Mercado libre realizando una matriz comparativa detallada entre ambas entidades para extraer mejores prácticas aplicables a nuestro desarrollo. Él se encargó de investigar sobre Andreani y Correo Argentino, y yo de Mercado libre, después nos juntamos para ver que cosas se repetían e identificar fortalezas, debilidades, ventajas y desventajas del sector logístico.

A partir del análisis de Correo Argentino, Mercado Libre y Andreani, pudimos identificar diferentes funcionalidades utilizadas actualmente, como el registro de paquetes, el seguimiento de envíos, la gestión de clientes y la organización de entregas.

Tomando estas características como referencia, nuestro proyecto propone un sistema de logística que permite centralizar y relacionar toda la información necesaria para realizar un envío. Según nuestro diseño, se pueden registrar los clientes, empleados, empresas adheridas, productos. paquetes, vehículos y

12/08

Este día nuestro profesor Gabriel Tuinyla nos dio un listado de los requerimientos de sus materias para la siguiente pre entrega que será a fines de agosto específicamente el 30/08 .

Analisis de sistema:

- Camino crítico
- Caso de uso extendido
- Dfd ()
- Historia de uso
- Calculo de horas

Base de datos:

- Actualizar diccionario de datos
- Actualizar DER

Proyecto informático 2:

- Terminar mvp

Dados los requerimientos mi compañero Benjamin se encargó de establecer la tarea de cada uno de mis compañeros. En mi caso me tocó hacer el caso de uso extendido y el conteo de horas.

18/08

Se dio inicio formal a la planificación temporal del proyecto mediante la construcción del Diagrama de Gantt. El propósito de este artefacto es establecer la previsión cronológica, la secuenciación de las tareas y la asignación de recursos para el desarrollo del software logístico. Para su elaboración, se adoptó un enfoque metodológico híbrido utilizando la herramienta Microsoft Excel para la matriz de tiempos y la plataforma Draw.io para la representación gráfica del mapa de dependencias.

22/08

Se llevó a cabo una sesión de control y seguimiento sobre el avance del Diagrama de Gantt en colaboración con el compañero Benjamin Saragusti. Durante esta jornada, se analizaron las dependencias lógicas entre las tareas de desarrollo y se ajustaron los tiempos estimados de los módulos de software. Esta revisión intermedia permitió actualizar la matriz de tiempos en la hoja de cálculo de Microsoft Excel, asegurando que el cronograma refleje de manera fiel el progreso del equipo antes de proceder con el modelado gráfico final.

26/08

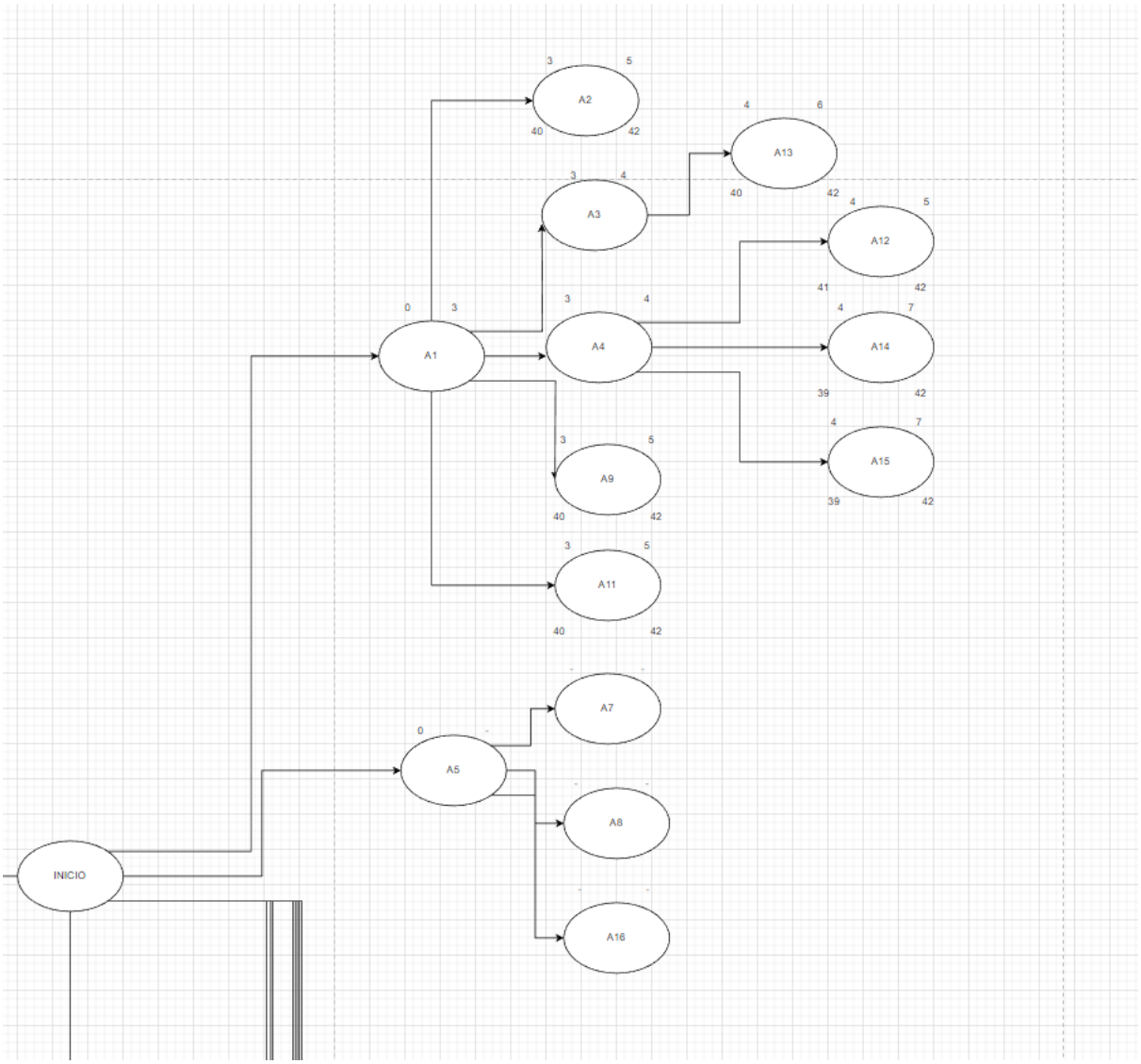
Se concluyó de forma exitosa el desarrollo del Diagrama de Gantt y la determinación analítica del Camino Crítico (CPM) del proyecto. A través de la herramienta Draw.io, se modeló visualmente la red de actividades indispensables y sus respectivas dependencias temporales. Este análisis permitió identificar con absoluta precisión la secuencia de tareas críticas sin margen de holgura, las cuales dictaminan la duración total del proyecto de software y garantizan la entrega a término de los entregables del sistema.

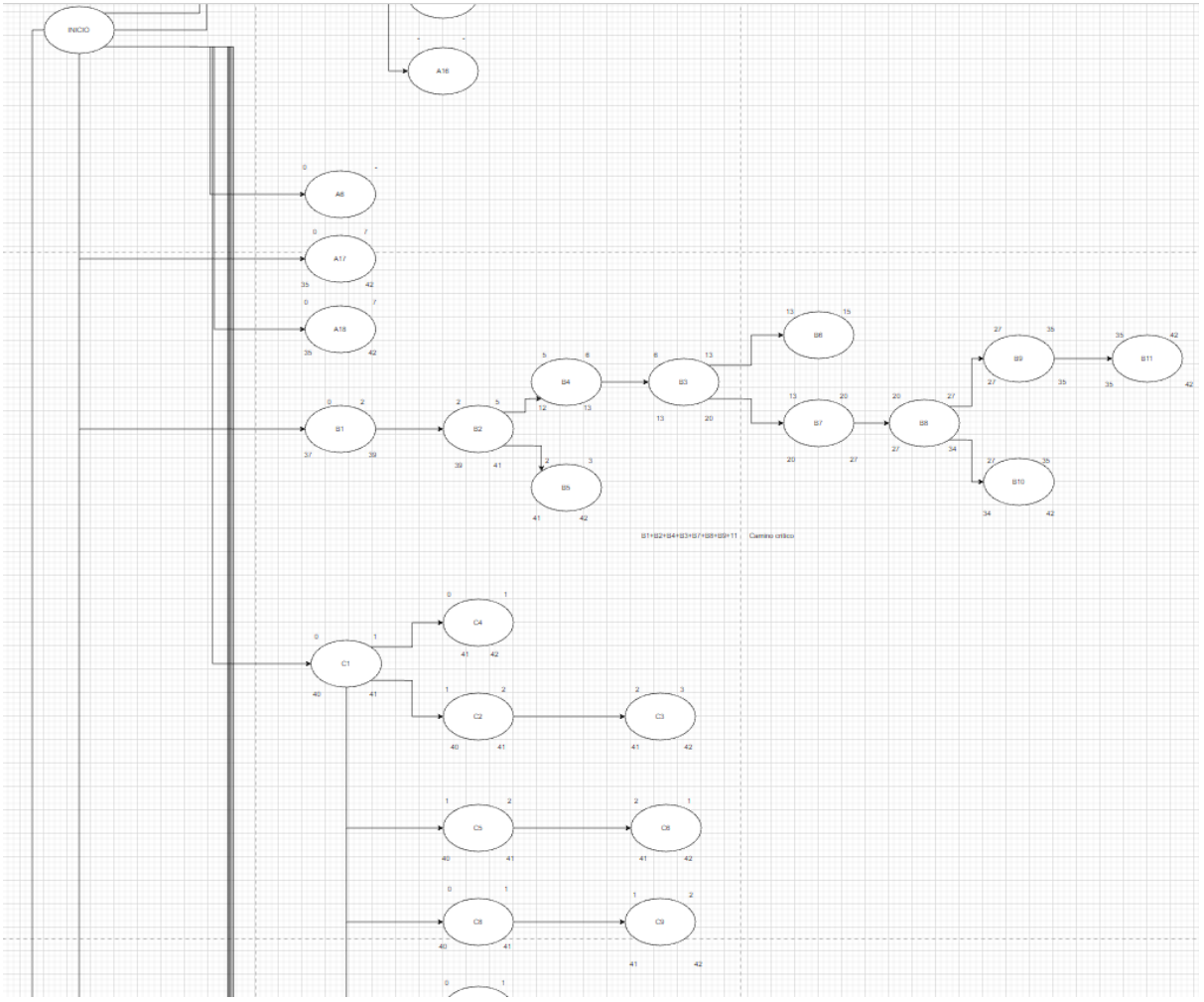
I	Simbolo	Dependiente de:		DIAS:
Relevamiento	A1	-		3
Transcribir relevamiento	A2		A1	2
Analisis foda	A3		A1	1
Analisis requerimientos	A4		A1	1
Diagrama gantt	A5	-		-
Diagrama pert	A6	-		-
Diagrama CPM Texto	A7		A5	1
Diagrama CPM Dibujito	A8		A5	2
Diagrama EDT	A8	-		1
Documentacion presentacion problematica y solucion	A9		A1	2
Documento estado de arte	A10	-		2
MVP:Minimo producto viable	A11		A1	2
Analisis tecnologico	A12		A4	1
DFD	A13		A3	2
Caso de uso extendido	A14		A4	3
Historia de usuario	A15		A4	3
Calculo de horas trabajados	A16		A5	3
Manual De uso	A17	-		7
Manual de explotacion	A18	-		7
Base de datos				
Diccionario de datos	B1	-		2
Der	B2		B1	3
Codificacion	B3		B4	7
Tablas	B4		B2	1
Relaciones	B5		B2	1
Valores	B6		B3	2
Funciones	B7		B3	7
Vistas	B8		B7	7

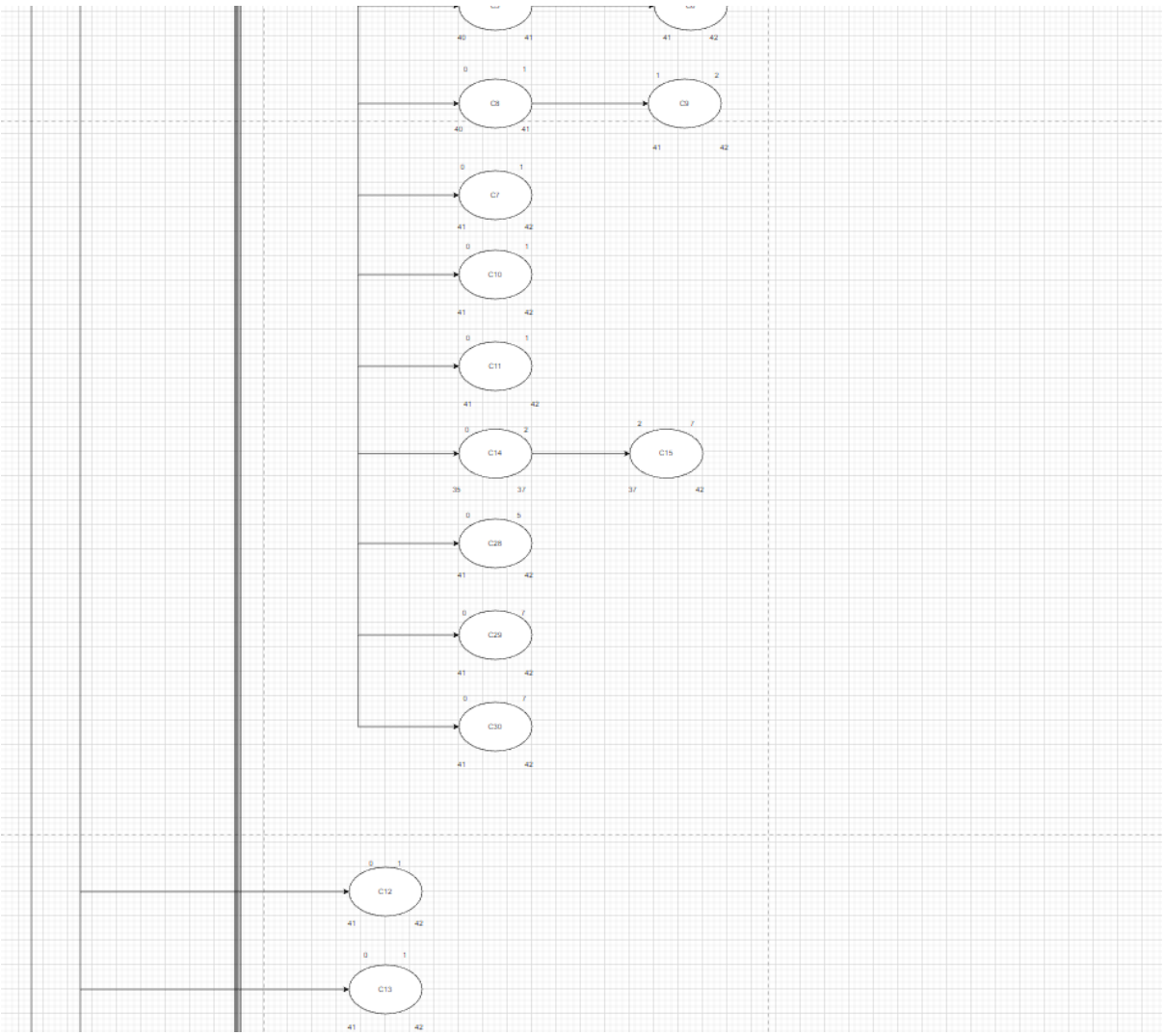
Vistas	B8		B7	7
Procedimientos	B9		B8	7
Triggers	B10		B8	7
Transacciones	B11		B9	7
Eliminacion logica,despliegue de motor	B12	-		7
Poo				
Identificar clases	C1	-		1
Padre:Persona	C2		C1	1
Hijas:Cliente,Proveedor,Transportista,Analista,Seguimiento	C3		C2	1
Paquete	C4		C1	1
Padre:Producto	C5		C1	1
Hijas:Tecnologico,Decoramiento,Indumentaria,Accesorios	C6		C5	1
Envio	C7		C1	1
Padre Vehiculo	C8		C1	1
Hijas:Avion,Barco,Tren,Camion	C9		C8	1
Empresa	C10		C1	1
EmpresasAdheridas	C11		C1	1
Definir atributos	C12	-		1
Metodos	C13	-		1
UML 1.0	C14		C1	2
Codificar	C15		C14	5
Padre:Persona	C16	-		1
Hijas:Cliente,Proveedor,Transportista,Analista,Seguimiento	C17	-		1
Paquete	C18	-		1
Padre:Producto	C19	-		1
Hijas:Tecnologico,Decoramiento,Indumentaria,Accesorios	C20	-		1
Envio	C21	-		1

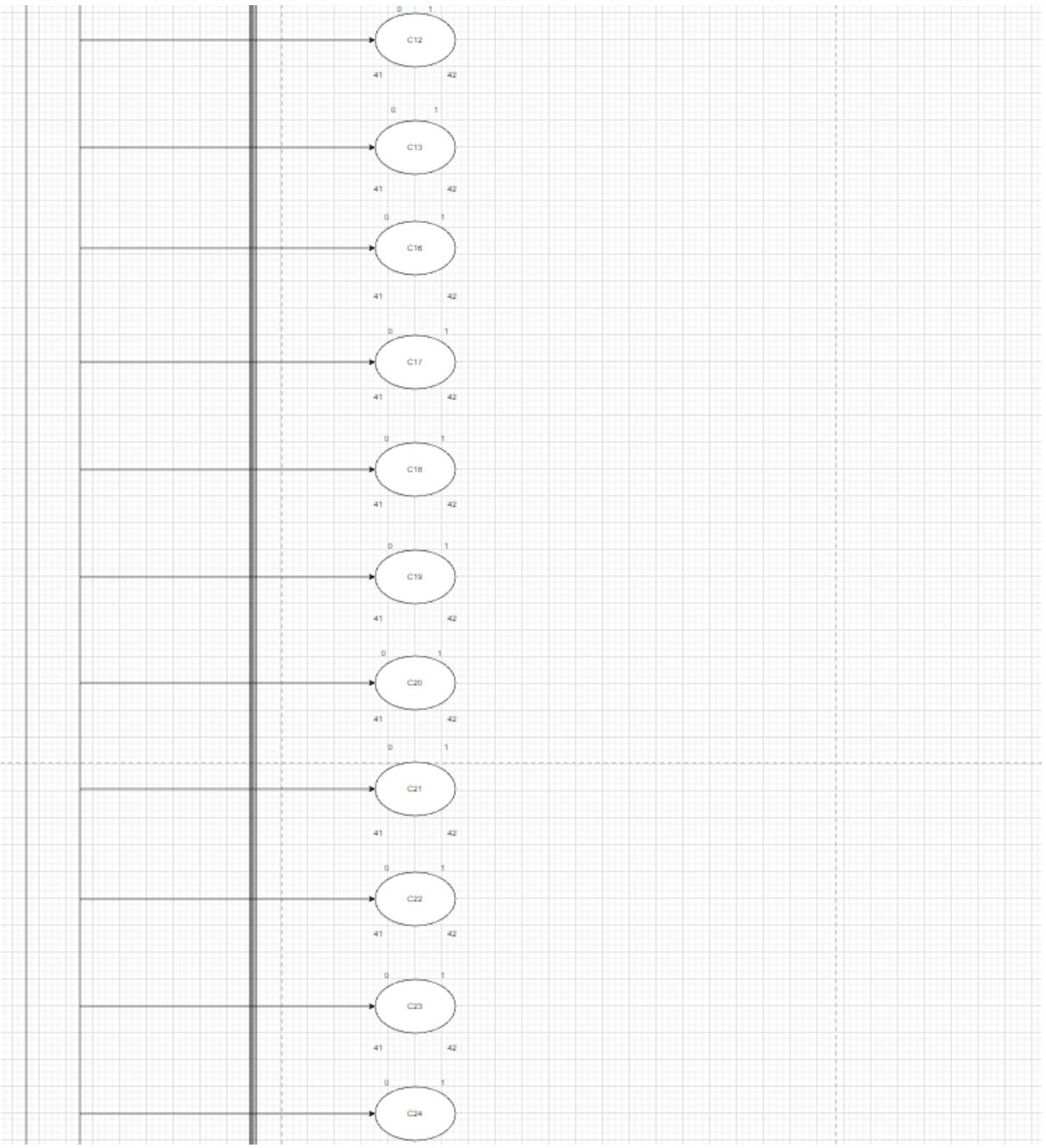
Padre Vehiculo	C22	-		1
Hijas:Avion,Barco,Tren,Camion	C23	-		1
Empresa	C24	-		1
EmpresasAdheridas	C25	-		1
Sacar clases hijas persona, vehiculo, producto	C26	-		2
Agregar clases administrador, ticket y reporte	C27	-		1
Modificar codigo java	C28		C1	5
Main	C29		C1	7
Interfaz grafica	C30		C1	7
Conexion con base de datos	C31	-		7
	C32			
Proyecto informatico 2				
Boceto (Sin funcionalidades)	D1	-		3
Definir funciones	D2	-		7
Logica de funcionamiento	D3	-		7
Separacion en paginas	D4		D3	7
Retoques artisticos	D5	-		7
vistas funcionales(minimo 10)	D6		D4	7
Vistas dinamicas	D7		D6	7
Conexion api http	D8	-		7
Diseño ui/ux correcto	D9		D5	7
Administracion y usuario	D10	-		7
Redes				
Identificar equipos	E1	-		1
Identificar cableado	E2	-		1
identificar Switches	E3	-		1
Identificar Routers	E4	-		1
Buscar precios	E5		E1-E4	1
Calcular presupuestos	E6		E5	1
Encontrar correlaciones en Cisko	E7	.-		1
Armar red en cisko (Profundizacion abajo)	E8	-		7
Colocar Dispositivos	E9	-		2
Conectarlos adecuadamente	E10	-		1
Configurar routers	E11	-		2
Comprobar conectividad con pings	E12	-		1
Guardar resultados	E13	-		1
Servidores	E14		E11	2
Ips estatico y dinamico	E15		E14	2

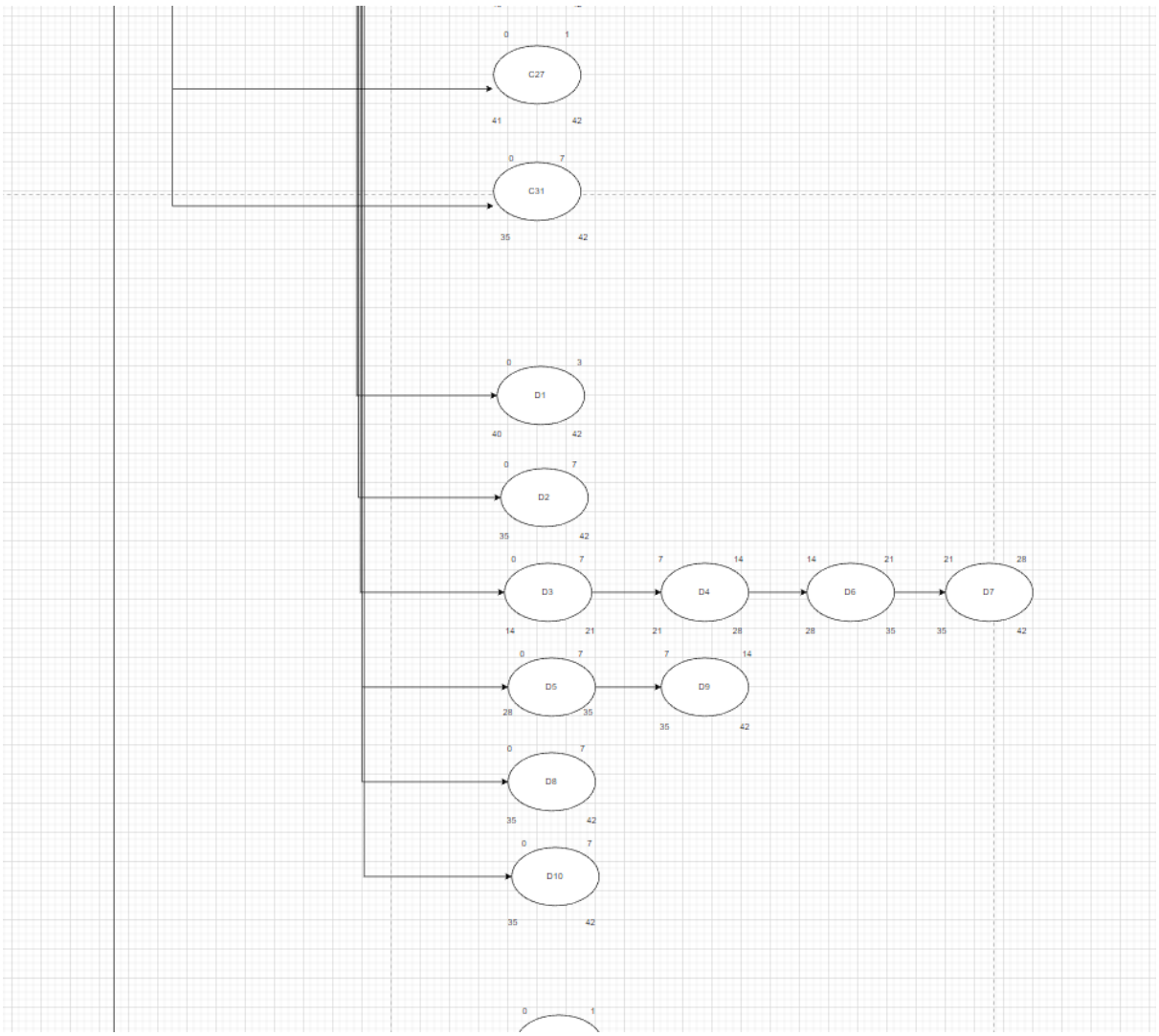
Tambien realize el camino critico mediante draw.io

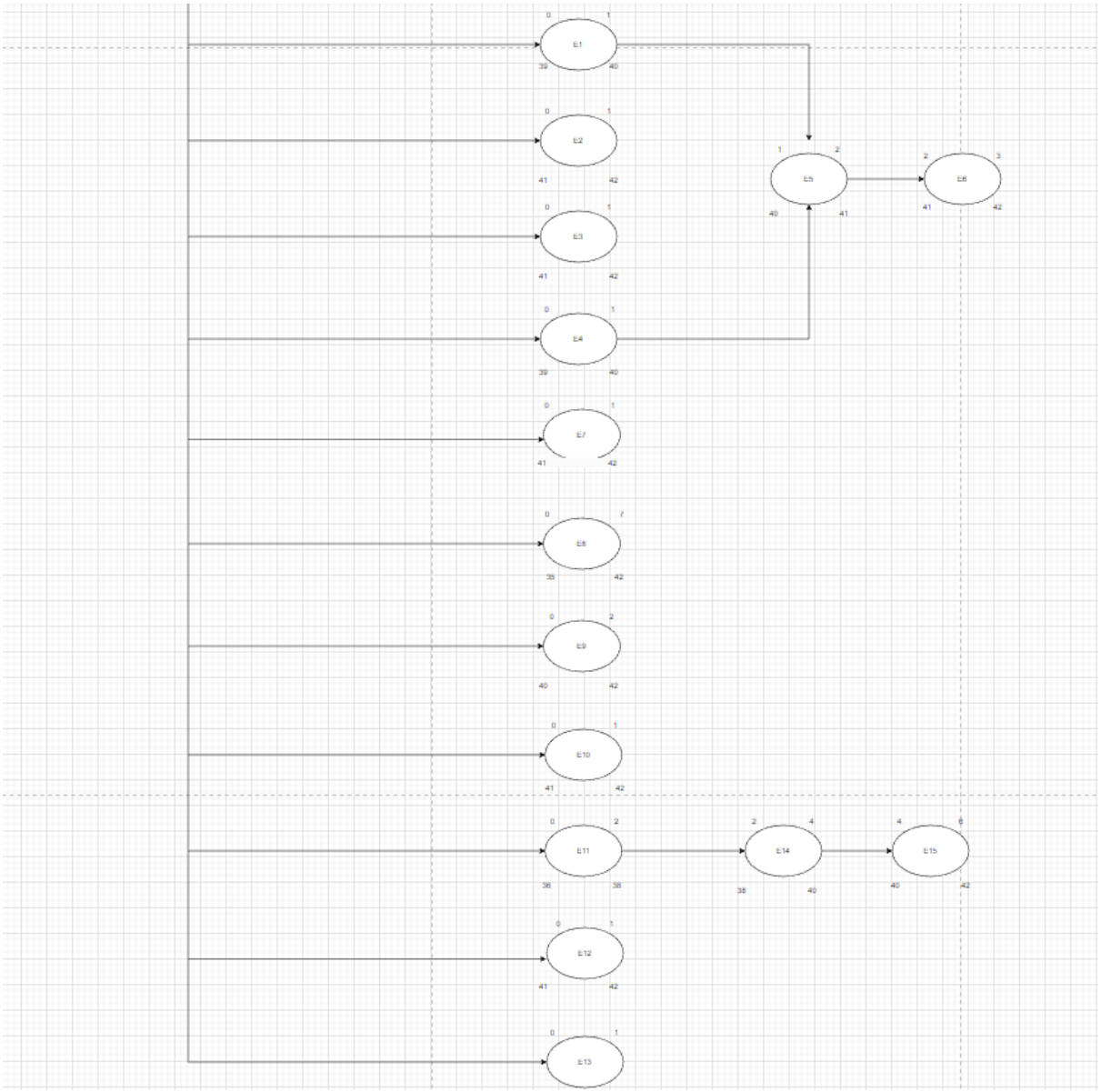

INICIO
A1
0
3
A2
3
5
40
42
A3
3
4
A4
3
4
A5
0
A7
A8
A9
3
5
40
42
A11
3
5
40
42
A12
4
5
41
42
A13
4
6
40
42
A14
4
7
39
42
A15
4
7
39
42
A16


INICIO
A16
A6
A17
A18
B1
B2
B4
B3
B6
B7
B8
B9
B11
B5
B10
B1+B2+B4+B3+B7+B8+B9+11 Camino crítico
C1
C4
C2
C3
C5
C6
C8
C9


C8
C9
C7
C10
C11
C14
C15
C28
C29
C30
C12
C13


C12
C13
C16
C17
C18
C19
C20
C21
C22
C23
C24


0
1
C27
41
42
0
7
C31
35
42
0
3
D1
40
42
0
7
D2
35
42
0
7
D3
14
21
7
14
D4
21
28
14
21
D6
28
35
21
28
D7
35
42
0
7
D5
28
35
7
14
D9
35
42
0
7
D8
35
42
0
7
D10
35
42
0
1



28/08

Se realizó la etapa de diseño teórico y el planteo estructurado de los componentes avanzados para la base de datos relacional. Se definieron las especificaciones iniciales para la creación de Procedimientos Almacenados (Stored Procedures), Vistas (Views), control de Transacciones (Transactions) y Disparadores (Triggers), orientados a garantizar la integridad y eficiencia de los datos.

31/08

Se llevó a cabo el planteo lógico y diseño estructural de los componentes esenciales para la gestión operativa de la base de datos, dividiendo los requerimientos en procedimientos almacenados para la manipulación de datos y vistas para la extracción automatizada de información:

- Modelado de Procedimientos Almacenados (*Stored Procedures*):
    1. registrar_nuevo_pedido: Centraliza la recepción de solicitudes de los clientes, automatizando la apertura del flujo logístico.
    2. asignar_pedido_transporte: Resuelve la asignación física de la carga vinculando un pedido específico con su chofer y vehículo correspondiente.
    3. actualizar_estado_entrega: Permite la actualización paramétrica de los estados del paquete para asegurar la trazabilidad en tiempo real.
- Modelado de Vistas Estructuradas (*Views*):
    1. vista_pedidos_pendientes: Consolida los registros de órdenes activas que requieren preparación física o despacho inmediato.
    2. vista_entregas_por_chofer: Agrupa el histórico y rendimiento de despachos completados por operario para auditoría de productividad.
    3. vista_vehiculos_disponibles: Filtra en tiempo real las unidades de la flota que se encuentran libres de rutas asignadas y operativas.

01/09

Se dio continuidad y cierre técnico a las tareas críticas de automatización e integridad referencial encomendadas por el líder del grupo Goldfard. Se establecieron las especificaciones para los bloques transaccionales y los disparadores reactivos que protegerán la consistencia lógica del inventario:

- Especificación de Bloques Transaccionales (Propiedades ACID):
    1. Transacción de Venta e Inventario: Encapsula de forma atómica el registro del pedido y la disminución física de las

unidades del stock; ante cualquier inconsistencia de saldo, ejecuta un ROLLBACK.
    2. Transacción de Reversa por Cancelación: Restablece las condiciones iniciales ante la anulación de una orden, reincorporando la mercadería al depósito de origen.
    3. Transacción de Movimiento Inter-Depósito: Coordina de manera simultánea el egreso de stock en un nodo logístico emisor y el ingreso en el nodo receptor, evitando pérdidas de mercadería en tránsito.
- Especificación de Disparadores (*Triggers*):
    1. tg_descontar_stock_pedido: Actúa de forma automática (AFTER INSERT) para deprimir las existencias físicas del producto al momento de anexarlo a una orden logística.
    2. tg_reponer_stock_devolucion: Automatiza el incremento del stock disponible en el instante en que se procesa el ingreso formal de un rechazo o devolución.
    3. tg_actualizar_estado_pedido: Escucha las modificaciones en el módulo de entregas y, al confirmarse la recepción conforme del cliente, conmuta el estado global de la orden a "Finalizado"

02/09

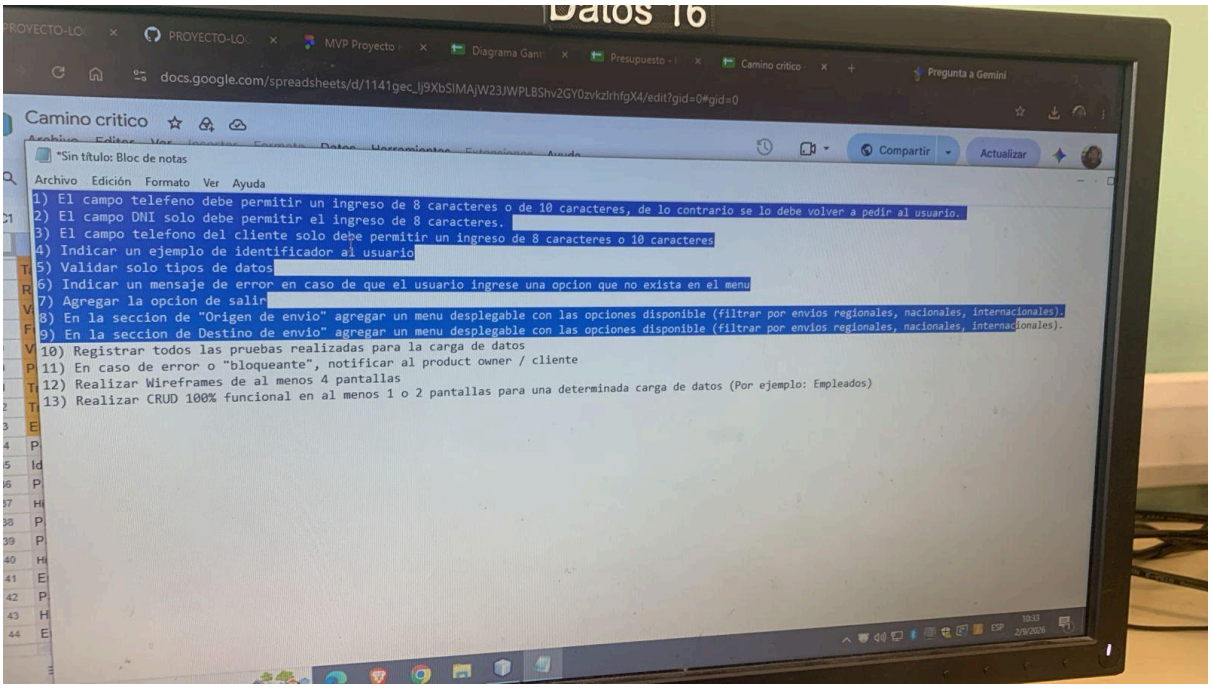
En esta jornada nos tocó afrontar una nueva instancia de pre entrega para mostrar todo el avance que veníamos acumulando hasta la fecha. Esta presentación ya venía con una serie de requisitos e indicaciones previas que nos habían pedido los profesores Nahuel Acosta y Gabriel Tuinyla. La razón por la cual tuvimos que exponer en este momento se debió a que, en la fecha anterior del 31 de agosto, el tiempo no alcanzó y los profesores

solamente llegaron a revisar las carpetas de los tres primeros grupos de la lista, quedando el nuestro para el 2/9. Cuando llegó nuestro turno de pasar a la corrección y nos evaluaron el material, los profesores nos marcaron varios puntos flojos y errores de debíamos solucionar:

- Con respecto al MVP: Nos comentaron que el diseño de las pantallas y la interfaz todavía se veía demasiado informal y que le faltaba una estética más pulida profesional.
- Sobre el DER (Diagrama entidad relación): Nos dijeron que había que rehacerlo directamente por completo porque visualmente era un caos, estaba muy desorganizado y encima habían quedado varias tablas repetidas que hacian ruido.
- En el diccionario de datos: Nos avisaron que había quedado desactualizado en comparación con los últimos cambios que hicimos, así que nos tocó ponernos a revisarlo para ponero al dia.
- En la parte de POO(Programación Orientada a Objetos): También nos encontraron cosas para corregir y nos dejaron apuntadas varias tareas y modificaciones que debíamos implementar en el código.

Una vez terminada la pre entrega, nos reunimos virtualmente para ver cómo dividimos el trabajo. Nuestro líder de equipo, Golfard, analizó la situación y nos asigno a Xiomara y a mí la tarea de encargarnos de todas

las correcciones correspondientes al código de Java, que son justamente las que se pueden ver en la foto que adjunto acá abajo. Cabe aclarar un detalle muy importante, nos pusimos de acuerdo en que solamente íbamos a resolver los primeros 9 puntos de la lista, ya que las consignas restantes exigen usar herramientas y conceptos de programación que todavía no llegamos a ver en la materia de POO, por lo que era imposible avanzar con eso por el momento.



Este día solo llegamos a hacer los dos primeros puntos. Los cuales eran:

1) El campo teléfono debe permitir un ingreso de 8 caracteres, de lo contrario se lo debe volver a pedir al usuario.

2) El campo dni solo debe permitir el ingreso de 8 caracteres.

3) El campo de teléfono del cliente solo debe permitir un ingreso de 8

caracteres o 10 caracteres

```
}
if(dni.length()!=8){
    JOptionPane.showMessageDialog(null, "ERROR - El DNI debe tener solo 8 digitos");
    contador++;
    return;
}


// condicionales telefono
if(!telefono.matches("[0-9]+") && !telefono.equals("")){
    JOptionPane.showMessageDialog(null, "ERROR - El telefono solo puede contener numeros.");
    contador++;
    return;
}
if(telefono.length()!=11 && telefono.length()!=10){
    JOptionPane.showMessageDialog(null, "ERROR - El telefono debe contar con 10 u 11 digitos");
    contador++;
    return;
}
```

06/09

El viernes 6 de septiembre nos juntamos con xiomara para terminar de realizar lo pedido, nos quedaban pendientes 7 puntos los cuales eran:

4) Indicar un ejemplo de identificador al usuario.

5) Validar solo tipos de datos.

6) Indicar un mensaje de error en caso de que el usuario ingrese una opción que no exista en el menú.

7) Agregar la opción de salir.

8) En la sección de “Origen de envío” agregar un menú desplegable con las opciones disponibles (filtrar por envíos regionales, nacionales, internacionales).

9) En la sección de “Destino de envío” agregar un menú desplegable con las opciones disponibles (filtrar por envíos regionales, nacionales, internacionales).

Cuando llegamos al punto 8 nos dimos cuenta que lo de menú desplegables no lo sabíamos porque todavía no lo habíamos visto en clase. Nuestro compañero Benjamin por su parte investigó como armar los menús desplegables en Java, logró entender la lógica del código, se encargó de programarlo él mismo.

09/09/2026

Este dia tuvimos una charla con nuestros profesores en donde nos dieron los requisitos para la pre entrega de septiembre los cuales son:

- POO(Programación Orientada a Objetos): Todas las ventanas que vayamos a usar con CRUD funcional y contador de errores.
- Redes: Servidor FTP y Correo.
- Proyecto Informático: Pantallas hasta donde lleguemos
- Base de datos: Procedimientos.

Una vez finalizada la charla nuestro compañero Benjamin se encargó de asignarnos nuestras respectivas tareas, se nos asigno realizar tres procedures básicos (insertar, editar y eliminar) para las distintas tablas, siendo un total de 9 procedures..

La división fue dos tablas normales y una tabla intermedia por integrante. A mi se me asignaron las tablas “productos”, “productos_paquetes” y “paquetes”.

- producto: Al crear un producto se debe verificar que el cliente exista y no se pueden borrar tickets que están en la tabla intermedia, reportes_tickets.
- Reportes: Al crear un reporte se debe verificar que el cliente exista y no se pueden borrar reportes que están en la tabla intermedia, reportes_tickets.
- Reportes_tickets: Se debe verificar que exista ese reporte y ese ticket.

15 de Septiembre de 2026

Se avanzó con el diseño e implementación técnica de los procedimientos almacenados correspondientes a la capa de manipulación de datos para las tablas de Productos, Paquetes y la entidad relacional intermedia Productos_Paquetes. En esta fase, el desarrollo se centró en la optimización de las consultas estructuradas de inserción, actualización y filtrado paramétrico, asegurando que la comunicación entre el sistema de persistencia y la lógica del negocio fuera eficiente.

20 de Septiembre de 2026

Se finalizó con éxito el proceso de generación e inserción de datos de prueba en el entorno de persistencia relacional. Se completó de forma masiva un lote de 50 registros de inserción exhaustivos dentro de la tabla de Paquetes. Cada uno de estos registros fue testeado de manera individual y validado bajo las reglas de negocio y restricciones lógicas de integridad establecidas en el modelo del sistema logístico.

21 de Septiembre de 2026

Se dio continuidad y cierre formal a las tareas de poblamiento inicial de la base de datos relacional. Se concretó la inserción de 50 registros de prueba adicionales en las tablas de Productos y en la tabla intermedia Productos_Paquetes. Mediante este procedimiento, se logró consolidar un conjunto de datos robusto, consistente y con integridad referencial completa, indispensable para dar inicio a la posterior fase de auditoría y pruebas integrales del sistema informático.

23 de Septiembre de 2026

Se realizó una jornada de codificación colaborativa junto a los compañeros Santiago Pedraza y Xiomara Ureña sobre el módulo principal del backend de la aplicación. Durante esta sesión, se diseñó e implementó un mecanismo de seguridad para la autenticación de usuarios que limita de forma estricta la cantidad máxima de intentos de acceso permitidos antes

del bloqueo y cierre automático de la terminal del programa. Asimismo, se incorporó una rigurosa validación de tipos de datos y caracteres en las máscaras de entrada de los formularios de la interfaz de usuario, restringiendo el ingreso de datos únicamente al formato sintáctico correspondiente para mitigar fallas en tiempo de ejecución o inyecciones de código.

30 de Septiembre de 2026

Se dedicó la jornada técnica a la redacción, ordenamiento y estructuración formal de la Carpeta de Campo. Se recopilaron los diagramas, presupuestos, análisis de competencia y registros de actividades acumulados hasta el momento para conformar la entrega provisoria orientada a la supervisión, control de cambios y auditoría pedagógica del proyecto por parte del cuerpo docente.

2 de Octubre de 2026

Se inició una etapa de investigación técnica y relevamiento de herramientas de diseño para el modelado de la infraestructura física del proyecto logístico. Se evaluó el entorno de modelado tridimensional SketchUp, utilizando la inteligencia artificial Gemini como soporte interactivo para acelerar el aprendizaje de los comandos de la interfaz, el manejo de escalas y la renderización de los almacenes centrales de distribución.

1. El espacio de trabajo y la navegación

Al abrir un lienzo nuevo, verás tres líneas de colores (los ejes **rojo, verde y azul**) que representan el espacio tridimensional. La clave para no perderte es dominar el mouse: YouTube · Sección Fugada

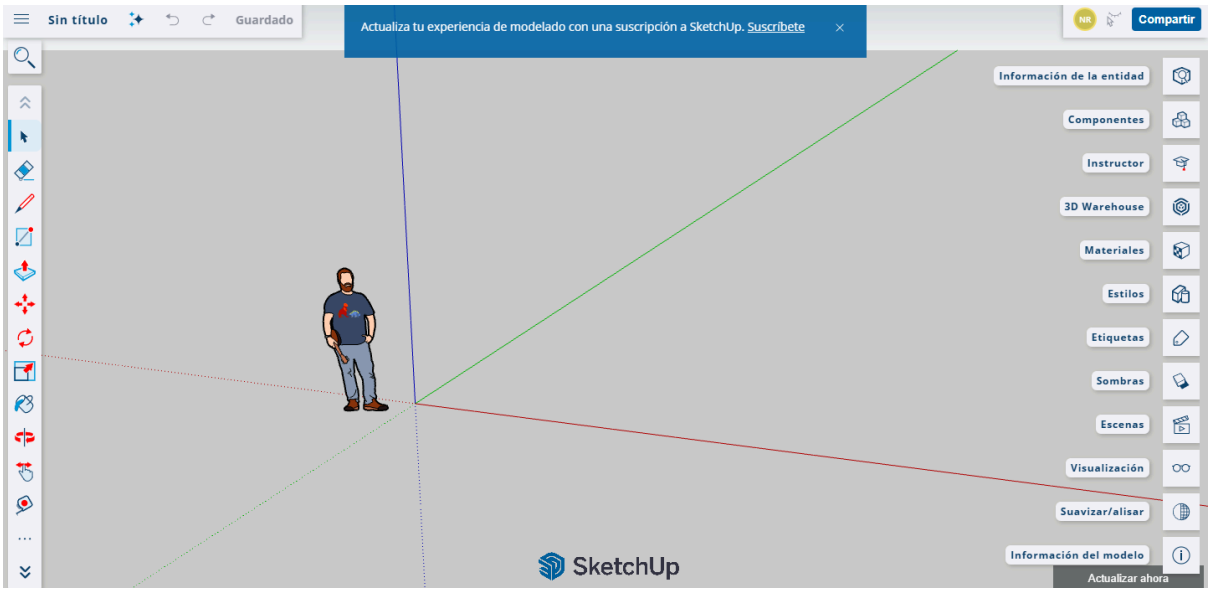
- **Zoom:** Gira la rueda del mouse hacia adelante para acercarte y hacia atrás para alejarte.
- **Orbitar (Girar la vista):** Mantén **presionada la rueda del mouse** y muévelo para rotar alrededor de tu objeto.
- **Desplazar (Moverse lateralmente):** Mantén presionada la tecla **Shift + la rueda del mouse** para mover la pantalla de lado a lado. YouTube · Un Arquitect... +2

---

2. Las herramientas esenciales de dibujo

En la barra de herramientas izquierda o superior encontrarás todo lo necesario para empezar a crear: Domestika +1

- **Seleccionar (Flecha negra / Barra espaciadora):** Sirve para tocar líneas, caras u objetos enteros.
- **Línea (Lápiz / Tecla L):** Dibuja tramos rectos. Al trazar, cuida que la línea cambie al color del eje (rojo, verde o azul) para asegurarte de que estás dibujando de forma derecha en el espacio tridimensional.



05/10

Se llevó a cabo el diseño final, depuración y codificación de los últimos procedimientos almacenados requeridos para el motor de la base de datos relacional. Durante esta jornada, las tareas se enfocaron en verificar la correcta implementación de las consultas estructuradas de manipulación de datos, asegurando la consistencia de las restricciones de integridad y la optimización de los tiempos de respuesta del servidor en el flujo de información del sistema logístico.

Agregar Productos:

```
DELIMITER //
CREATE PROCEDURE agregar_productos(
   IN marca VARCHAR(100),
   in descripcion varchar(200),
   in precio decimal (10,2),
   in stock int,
   in categoria varchar (50),
   OUT mensaje VARCHAR(200)
)
BEGIN
   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      SET mensaje = 'Error: operación revertida';
      ROLLBACK;
   END;

   START TRANSACTION;

   INSERT INTO productos( marca, descripcion,precio , stock,categoria )
   VALUES ( marca, descripcion, precio, stock, categoria);

   COMMIT;
   SET mensaje = 'Hecho con éxito';
END//
DELIMITER ;
```

utilización de la herramienta de la IA gemini para su corrección y comprobación

DELIMITER //
CREATE PROCEDURE agregar_productos(...

Sí, el código está perfectamente estructurado y es sintácticamente correcto para MySQL. Cumple muy bien con las buenas prácticas al implementar transacciones y control de excepciones (`EXIT HANDLER`).

Sin embargo, hay un detalle crítico en los nombres de las variables que podría causar que **el procedimiento inserte valores vacíos o devuelva un error en algunas versiones de MySQL**:

⚠ El problema de la "Ambigüedad de Nombres"

En MySQL, si una variable o parámetro de entrada (`IN`) se llama exactamente igual que una columna de la tabla, el motor puede confundirse. En tu línea:

sql

```
VALUES ( marca, descripcion, precio, stock, categoria);
```

Usa el código con precaución.

codigo:
Eliminar productos:

```
delimiter //
create procedure borrar_producto(in p_producto int , out mensaje
varchar(100))
        BEGIN
        DECLARE v_producto int default 0;
      start transaction;
      select count(id_producto) into v_producto
      from productos_paquetes
      where productos_paquetes.ID_Producto = p_producto;

      if v_producto > 0 THEN
```

```
        set mensaje="El producto esta en un paquete";
          ROLLBACK;
       else
        delete from productos
          where id_producto = p_producto;
          COMMIT;
          set mensaje = 'Producto Eliminado con Éxito.';
       end if;
end//
delimiter;
```

Comprobación del mismo

Sí, la lógica de tu código está bien pensada y es segura, ya que usas una transacción y verificas de forma excelente que el producto no esté amarrado a un paquete antes de intentar borrarlo.

Sin embargo, contiene **un error de sintaxis al final que hará que falle al ejecutarlo**, y un par de detalles de consistencia con las transacciones que deberías ajustar.

Aquí tienes el análisis de los detalles y el código corregido:

6/10

seguimiento de los procedimientos de base de datos

codigo:
Editar Productos :

```
DELIMITER //

CREATE PROCEDURE editar_productos(
```

```
    IN p_id_producto INT,
    IN p_Marca VARCHAR(200),
    IN p_Descripcion VARCHAR(200),
    IN p_Precio DECIMAL(10,2),
    IN p_Stock INT,
    IN p_Categoria VARCHAR(200),
    OUT p_mensaje VARCHAR(200)
)
BEGIN
    DECLARE auxiliar VARCHAR(100);

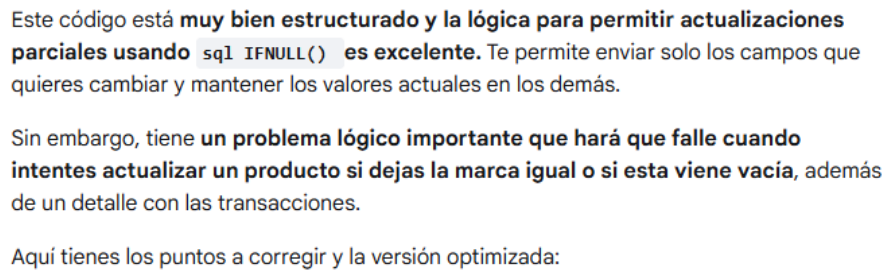
    START TRANSACTION;
        SELECT productos.Marca INTO auxiliar FROM productos WHERE
productos.ID_Producto = p_id_producto;

        IF auxiliar IS NULL THEN
            SET p_mensaje = 'No existe el producto';
            ROLLBACK;
        ELSE
            UPDATE productos
            SET
                Marca = IFNULL(p_Marca, Marca),
                Descripcion = IFNULL(p_Descripcion, Descripcion),
                Precio = IFNULL(p_Precio, Precio),
                Stock = IFNULL(p_Stock, Stock),
                Categoria = IFNULL(p_Categoria, Categoria)
            WHERE ID_Producto = p_id_producto;

            COMMIT;
            SET p_mensaje = 'Producto actualizado correctamente';
        END IF;
END //

DELIMITER ;
```

comprobación



Este código está **muy bien estructurado y la lógica para permitir actualizaciones parciales usando** `sql IFNULL()` **es excelente.** Te permite enviar solo los campos que quieres cambiar y mantener los valores actuales en los demás.

Sin embargo, tiene **un problema lógico importante que hará que falle cuando intentes actualizar un producto si dejas la marca igual o si esta viene vacía**, además de un detalle con las transacciones.

Aquí tienes los puntos a corregir y la versión optimizada:

codigo:

Agregar Productos_Paquetes:

```
DELIMITER //
CREATE PROCEDURE agregar_productos_paquetes(
    IN ID_Producto int,
    in ID_Paquete int,
    in cantidad int,
    OUT mensaje VARCHAR(200)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET mensaje = 'Error: operación revertida';
        ROLLBACK;
    END;

    START TRANSACTION;

    INSERT INTO productos_paquetes ( ID_Producto, ID_Paquete
,cantidad)
    VALUES (ID_Productos, ID_Paquetes ,cantidad);
```

```
    COMMIT;
    SET mensaje = 'Hecho con éxito';
END//
DELIMITER ;
```

Este procedimiento tiene la estructura correcta y el manejo de excepciones está muy bien planteado, pero **va a fallar inmediatamente al ejecutarlo debido a errores en los nombres de las variables**.

Tiene exactamente los mismos dos detalles que corregimos en tus primeros procedimientos: **errores de dedo (typos)** en los nombres de las variables del `INSERT` y la **ambigüedad de nombres** con las columnas de tu tabla.

⚠️ **Errores específicos a corregir:**

1. **Variables mal escritas (Plural vs. Singular):** En la declaración pusiste `IN ID_Producto` e `IN ID_Paquete`. Sin embargo, abajo en el `VALUES` escribiste `ID_Productos` e `ID_Paquetes` (con **S** al final). MySQL te dará un error diciendo que esas variables no existen.
2. **Ambigüedad de nombres:** Al llamarse tus parámetros exactamente igual que las columnas de la tabla (`ID_Producto`, `ID_Paquete`, `cantidad`), MySQL se va a confundir y podría intentar insertar el valor de la columna en sí misma.

Corrección

```
DELIMITER //

CREATE PROCEDURE agregar_productos_paquetes(
    IN p_id_producto INT,
    IN p_id_paquete INT,
    IN p_cantidad INT,
    OUT mensaje VARCHAR(200)
)
BEGIN
     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET mensaje = 'Error: operación revertida. Verifique que el producto y el paquete existan.';
        ROLLBACK;
```

```
    END;

    START TRANSACTION;


    INSERT INTO productos_paquetes (ID_Producto, ID_Paquete,
cantidad)
    VALUES (p_id_producto, p_id_paquete, p_cantidad);

    COMMIT;
    SET mensaje = 'Hecho con éxito';
END//

DELIMITER ;
```

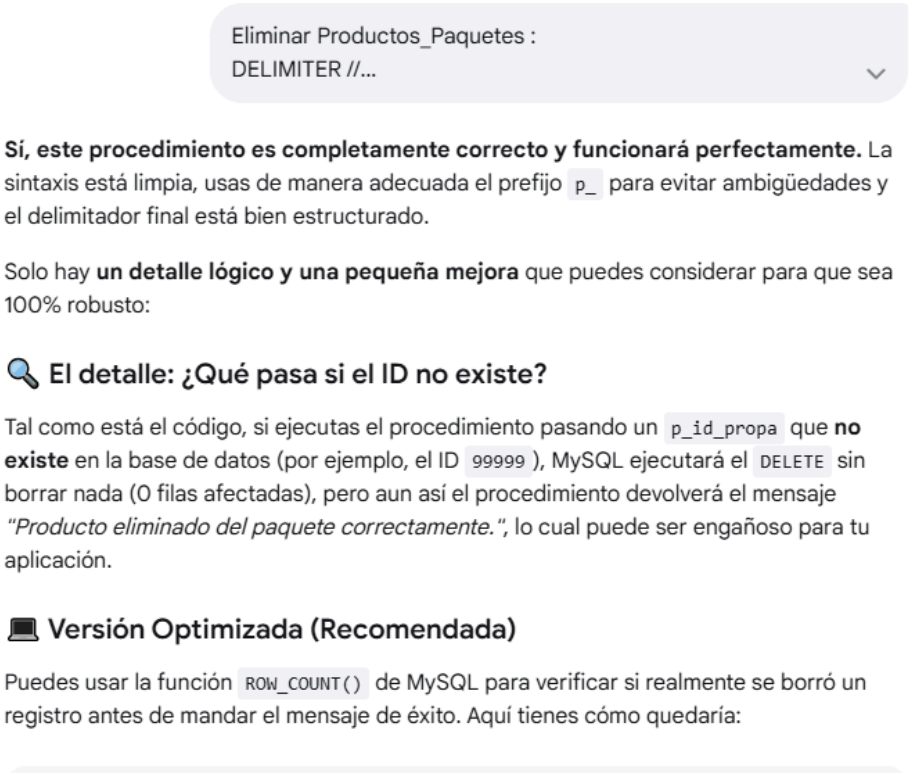
codigo:

Eliminar Productos_Paquetes :

```
DELIMITER //
CREATE PROCEDURE borrar_producto_paquete(IN p_id_propa INT,
OUT mensaje VARCHAR(100))
BEGIN
        DELETE FROM Productos_Paquetes
        WHERE ID_Propa = p_id_propa;
        SET mensaje = 'Producto eliminado del paquete correctamente.';
END //

DELIMITER ;
```

Comprobacion:



07/10

El líder del equipo técnico, Santiago Goldfard, coordinó la asignación de tareas correspondientes al desarrollo de las interfaces gráficas de usuario para el módulo de Programación Orientada a Objetos. Mi contribución específica en esta etapa consistió en asumir la responsabilidad del diseño, maquetación y codificación de las ventanas operativas para los módulos de Empresas Adheridas y Control de Vehículos de la flota logística.

8/10

Se dio continuidad al desarrollo y programación de la capa de persistencia en la base de datos relacional. Durante esta jornada, me enfoqué en la

creación de rutinas avanzadas de consulta mediante procedimientos almacenados.

Buscar productos por fecha de ensamble

```
drop procedure if exists buscar_producto_fecha_ensamble;
DELIMITER //
create procedure buscar_producto_fecha_ensamble(in fecha_ensamble_ing
varchar(100))
begin
select * from fecha_ensamble
where Precio = fecha_ensamble_ing ;
end //
DELIMITER ;
```

Buscar productos por precio

```
drop procedure if exists buscar_producto_por_precio;
DELIMITER //
create procedure buscar_producto_por_precio(in precio_ing varchar(100))
begin
select * from producto
where Precio = precio_ing;
end //
DELIMITER ;
```

Se concluyó de forma exitosa la programación, testeo local y finalización de la totalidad de los procedimientos almacenados que habían sido solicitados por el líder del equipo de desarrollo, logrando la estabilización de las consultas requeridas para la persistencia del sistema antes de pasar a las fases de integración con la interfaz gráfica.

11/10- 17/10

Se dio inicio al desarrollo e implementación de las interfaces gráficas de usuario correspondientes a los módulos asignados, específicamente las ventanas para la gestión de Empresas Adheridas. Durante este período, las tareas se centraron en la codificación de la lógica visual y de eventos en el entorno de desarrollo de Java.

En paralelo, se trabajó en el modelado tridimensional de la infraestructura logística utilizando la herramienta SketchUp. Debido a la complejidad técnica del diseño tridimensional de los almacenes y flotas, se recurrió al uso de la inteligencia artificial Gemini como soporte pedagógico y técnico. Esto permitió agilizar la curva de aprendizaje, optimizar el uso de las herramientas de geometría y lograr un prototipo a escala que representa de manera fiel la distribución espacial de la empresa.

```java
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package ejercicio;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
/**
 *
 * @author Benjamin
 */
public class AdheridoDAO {
    public void insertar(Empresas_Adheridas c) throws SQLException {
    String consulta_sql = "CALL agregar_empresa_adherida(?, ?, ?, ?, ?, ?, ?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setString(1, c.getNombre());
        sentencia_sql.setString(2, c.getCuil());
        sentencia_sql.setString(3, c.getDireccion());
        sentencia_sql.setInt(4, c.getTelefono());
        sentencia_sql.setString(5, c.getRubro());
        sentencia_sql.setString(6, c.getDuenio());

        sentencia_sql.registerOutParameter(7, java.sql.Types.VARCHAR);

        sentencia_sql.executeUpdate();

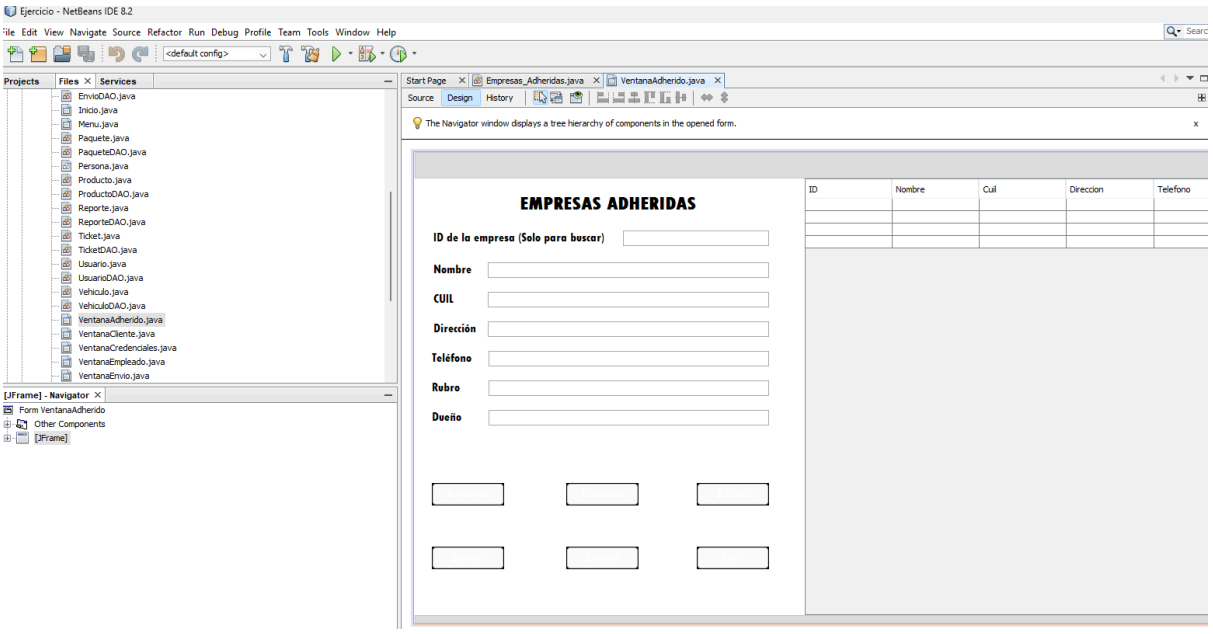
        String mensajeRespuesta = sentencia_sql.getString(7);
        System.out.println("Respuesta del servidor: " + mensajeRespuesta);
        System.out.println("Empleado guardado exitosamente");

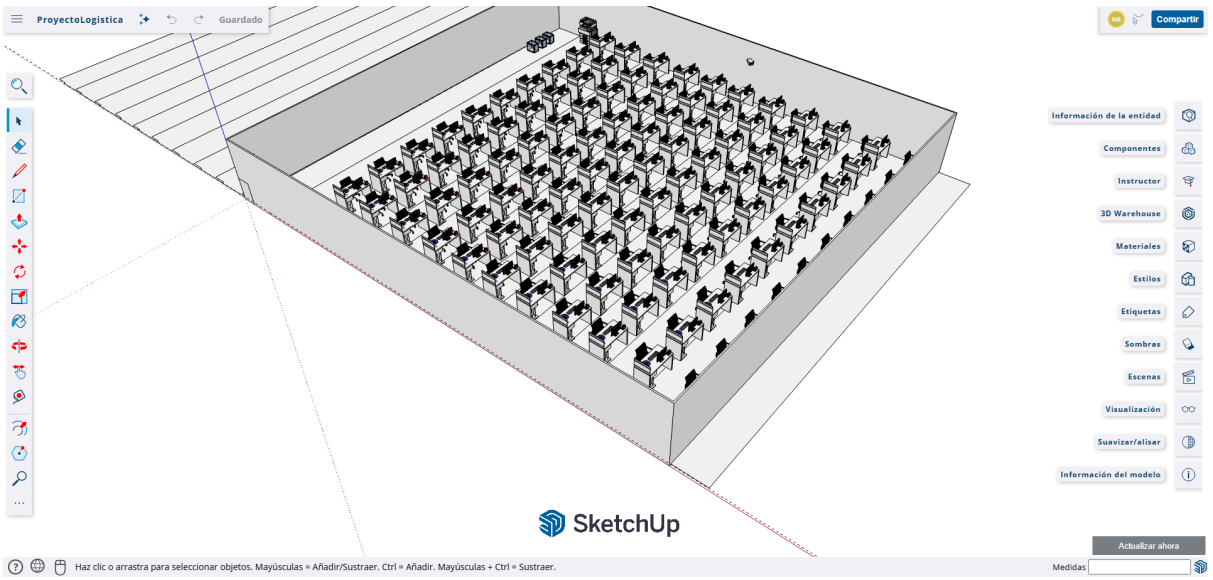
    } catch (SQLException ex) {
        System.out.println("ERROR :" + ex.getMessage());
```

Start Page | Empresas_Adheridas.java | VentanaAdherido.java | AdheridoDAO.java | VentanaVehiculo.java

Source | History

```
40          } catch (SQLException ex) {
41              System.out.println("ERROR :" + ex.getMessage());
42          }
43      }
44      public  ArrayList<Empresas_Adheridas> listar(){
45          ArrayList<Empresas_Adheridas> adherido=new ArrayList<>();
46          String consulta_sql="call mostrar_adheridos()";
47
48              Connection conexion=Conexion.conectar();
49              try {
50                  CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
51                  ResultSet resultado=sentencia_sql.executeQuery();
52                  while(resultado.next()){
53                      Empresas_Adheridas p=new Empresas_Adheridas();
54                      p.setId_empresa(resultado.getInt("ID_empresa"));
55                      p.setNombre(resultado.getString("nombre"));
56                      p.setCuil(resultado.getString("cuit"));
57                      p.setDireccion(resultado.getString("direccion"));
58                      p.setDuenio(resultado.getString("duenio"));
59                      p.setRubro(resultado.getString("rubro"));
60                      p.setTelefono(resultado.getInt("telefono"));
61                      adherido.add(p);
62                      System.out.println("------------------");
63                  }
64              } catch (SQLException ex) {
65                  System.out.println("ERROR "+ex.getMessage());
66              }
67              return adherido;
68
69
70          }
71      public Empresas_Adheridas buscar(int id) {
72       String consulta_sql = "call buscar_Adheridos(?)";
73       Connection conexion = Conexion.conectar();
74
75       try {
76           CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
77           sentencia_sql.setInt(1, id);
78           ResultSet resultado = sentencia_sql.executeQuery();
79
80           if (resultado.next()) {
```





18/10
Culminacion del objetivo planteado por el lider del grupo, finalizacion de la ventana Vehiculos

codigo:

```
package ejercicio;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

public class VehiculoDAO {

    public void insertar(Vehiculo v)throws SQLException {
        String consulta_sql = "CALL crear_vehiculo(?, ?, ?, ?, ?, ?)";

        try{
            Connection conexion = Conexion.conectar();
```

```
            CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);



            sentencia_sql.setInt(1, v.getPrecio());
            sentencia_sql.setInt(2, v.getAnio());
            sentencia_sql.setInt(3, v.getIdEmpleado());
            sentencia_sql.setString(4, v.getModelo());
            sentencia_sql.setString(5, v.getMarca());

            sentencia_sql.registerOutParameter(6, java.sql.Types.VARCHAR);
            sentencia_sql.executeUpdate();
            String mensajeRespuesta = sentencia_sql.getString(6);
            System.out.println("Respuesta del servidor: " +
mensajeRespuesta);
            System.out.println("Vehiculo guardado correctamente");
        }catch(SQLException ex){
            System.out.println("ERROR "+ ex.getMessage());
        }
    }

    public ArrayList <Vehiculo> listar(){
        ArrayList <Vehiculo> lista_vehiculo = new ArrayList<>();
        String consulta_sql="call mostrar_vehiculos()";
        try{
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);
            ResultSet resultado=sentencia_sql.executeQuery();

            while (resultado.next()){
                Vehiculo v = new Vehiculo();

                v.setIdvehiculo(resultado.getInt("ID_Vehiculo"));
                v.setPrecio(resultado.getInt("Precio"));
```

```
                v.setAnio(resultado.getInt("anio"));
                v.setIdEmpleado(resultado.getInt("ID_Empleado"));
                v.setModelo(resultado.getString("Modelo"));
                v.setMarca(resultado.getString("Marca"));




                lista_vehiculo.add(v);
                System.out.println("~~~~~~~~~~~~~~~~~~~");
            }
        }catch (SQLException ex){
            System.out.println("ERROR "+ ex.getMessage());
        }
        return lista_vehiculo;
    }

    public void actualizar (Vehiculo v){
         String consulta_sql = "call editar_vehiculos (?, ?, ?, ?, ?, ?, ?)";

        try {
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);

            sentencia_sql.setInt(1, v.getIdvehiculo());
            sentencia_sql.setInt(2, v.getPrecio());
            sentencia_sql.setInt(3, v.getAnio());
            sentencia_sql.setInt(4, v.getIdEmpleado());
            sentencia_sql.setString(5, v.getModelo());
            sentencia_sql.setString(6, v.getMarca());

            sentencia_sql.registerOutParameter(7, java.sql.Types.VARCHAR);
            sentencia_sql.executeUpdate();
            String respuesta = sentencia_sql.getString(7);
```

```
            System.out.println("Respuesta de la BD: " + respuesta);
            System.out.println("Datos actualizados exitoamnete");

        }catch (SQLException ex){
            System.out.println("ERROR "+ ex.getMessage());
        }

    }

    public Vehiculo buscar(int id) {
    String consulta_sql = "call buscar_vehiculo(?)";
    Connection conexion = Conexion.conectar();

    try {
        CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);
        sentencia_sql.setInt(1, id);
        ResultSet resultado = sentencia_sql.executeQuery();

        if (resultado.next()) {
            Vehiculo v =new Vehiculo();
                v.setIdvehiculo(resultado.getInt("ID_Vehiculo"));
                v.setPrecio(resultado.getInt("Precio"));
                v.setAnio(resultado.getInt("anio"));
                v.setIdEmpleado(resultado.getInt("ID_Empleado"));
                v.setModelo(resultado.getString("Modelo"));
                v.setMarca(resultado.getString("Marca"));

            System.out.println("------------------");
            return v;
        } else {
            System.out.println("No se encontró ningún vehiculo con el ID: " +
id);
        }
    } catch (SQLException ex) {
```

```
            System.out.println("ERROR " + ex.getMessage());
        }
        return null;
    }

        public void eliminar(int id) {
        String consulta_sql = "call eliminar_vehiculo (?, ?)";

        try {
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql =
    conexion.prepareCall(consulta_sql);

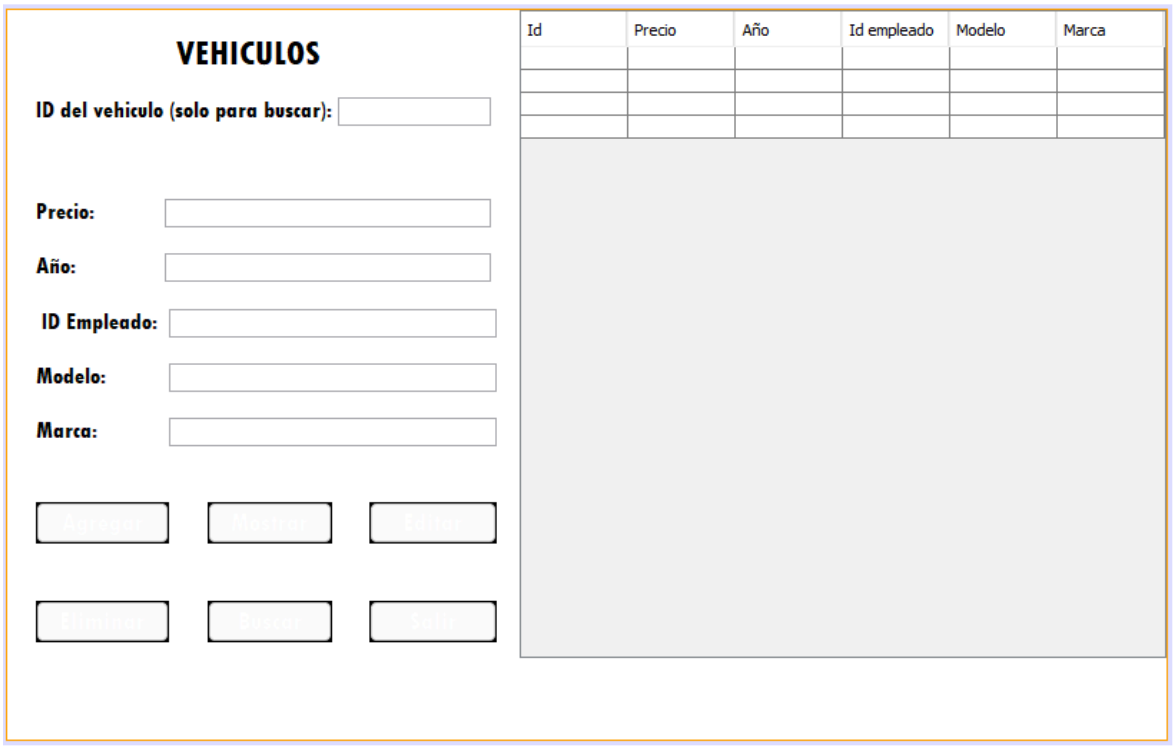
            sentencia_sql.setInt(1, id);

            sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);

            sentencia_sql.executeUpdate();

            String respuesta = sentencia_sql.getString(2);
            System.out.println("Respuesta de la BD: " + respuesta);
            System.out.println("Vehiculo eliminado");

        } catch (SQLException ex) {
            System.out.println("ERROR: " + ex.getMessage());
        }
    }
    }
```



20/10

Se llevaron a cabo las tareas de integración y establecimiento de conexiones entre la lógica de la aplicación en Java y las interfaces desarrolladas en HTML, trabajando de manera conjunta con la compañera Xiomara Ureña. Durante esta jornada de desarrollo colaborativo, se contó además con aportes técnicos y revisiones por parte de los compañeros Santiago Goldfard y Benjamin Saragusti, quienes colaboraron en la optimización del flujo de datos y la verificación de la comunicación.

codigo :

```
html lang="es">
<head>
   <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Editar paquete</title>

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/css/bootstr
ap.min.css" rel="stylesheet"
integrity="sha384-B/GM4XqrwHnWXNOWMbloTmrYXZg10cakYGmpf
sR/bbzQ6JAJI4ihuyADKLnBgrCe" crossorigin="anonymous">

    <script type="module"
src="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/js/bootstrap
.bundle.min.js"
integrity="sha384-1a/pXj49ZQ1aHEmrJ+gMw1otqoVsYwlEnlD8mIfY2T
V03r20Y0CN7uqx1tQogjPL" crossorigin="anonymous"></script>
    <link rel="stylesheet" href="general.css">
</head>

<body>

    <div class="fondo-blanco">

    <!-- BARRA DE BUSQUEDA -->
    <nav class="navbar fondo-blanco">
        <div class="container-fluid">
            <a class="navbar-brand texto-negro"
onclick="window.open('index.html')">

            Logistica
            </a>
```

```
          <li class="nav-item">
              <button type="button" class="btn hover-lift" aria-current="page"
href="#" onclick="window.open('VerEnvios.html', '_self');">
                <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-box" viewBox="0 0 16 16">
                <path d="M8.186 1.113a.5.5 0 0 0-.372 0L1.846 3.5 8 5.961
14.154 3.5zM15 4.239l-6.5 2.6v7.922l6.5-2.6V4.24zM7.5
14.762V6.838L1 4.239v7.923zM7.443.184a1.5 1.5 0 0 1 1.114 0l7.129
2.852A.5.5 0 0 1 16 3.5v8.662a1 1 0 0 1-.629.928l-7.185 2.874a.5.5 0 0
1-.372 0L.63 13.09a1 1 0 0 1-.63-.928V3.5a.5.5 0 0 1 .314-.464z"/>
                </svg>
                Ver envios</a>
          </li>
          <li class="nav-item">
              <button type="button" class="btn hover-lift"
onclick="alert('Numero de  telefono : +54 911 6767 6767 \nCorreo
electronico : logistica32@gmail.com  \nNombre de contacto:Logistica 32
Equipo 6')">
                <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-telephone" viewBox="0 0 16
16">
                <path d="M3.654 1.328a.678.678 0 0 0-1.015-.063L1.605
2.3c-.483.484-.661 1.169-.45 1.77a17.6 17.6 0 0 0 4.168 6.608 17.6 17.6 0
0 0 6.608 4.168c.601.211 1.286.033 1.77-.45l1.034-1.034a.678.678 0 0
0-.063-1.015l-2.307-1.794a.68.68 0 0 0-.58-.122l-2.19.547a1.75 1.75 0 0
1-1.657-.459L5.482 8.062a1.75 1.75 0 0 1-.46-1.657l.548-2.19a.68.68 0 0
0-.122-.58zM1.884.511a1.745 1.745 0 0 1 2.612.163L6.29
2.98c.329.423.445.974.315 1.494l-.547 2.19a.68.68 0 0 0 .178.643l2.457
2.457a.68.68 0 0 0 .644.178l2.189-.547a1.75 1.75 0 0 1 1.494.315l2.306
```

```
1.794c.829.645.905 1.87.163 2.611l-1.034 1.034c-.74.74-1.846
1.065-2.877.702a18.6 18.6 0 0 1-7.01-4.42 18.6 18.6 0 0
1-4.42-7.009c-.362-1.03-.037-2.137.703-2.877z"/>
                </svg>
                Contacto</button>
            </li>
            <button id="nombre_inicio" type="button" class="btn hover-lift"
aria-current="page" onclick="window.open('Inicio_sesion.html', '_self');">
              <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-person" viewBox="0 0 16
16">
              <path d="M8 8a3 3 0 1 0 0-6 3 3 0 0 0 0 6m2-3a2 2 0 1 1-4 0 2 2
0 0 1 4 0m4 8c0 1-1 1-1 1H3s-1 0-1-1 1-4 6-4 6 3 6
4m-1-.004c-.001-.246-.154-.986-.832-1.664C11.516 10.68 10.289 10 8
10s-3.516.68-4.168 1.332c-.678.678-.83 1.418-.832 1.664z"/>
              </svg>
              Iniciar sesion</button>
        </div>
    </nav>

    <div class="separador">

    </div>

    <h1 class="texto-negro"> Ingrese los datos del vehiculo </h1>

    <br>

      <div class="container mt-5">
      <div class="row justify-content-center">
```

```
      <div class="col-md-5 col-lg-3 test_form">
        <div class="card shadow">
          <div class="card-header bg-primary text-white">
            <h4 class="texto-formulario mb-0 text-center" >Formulario de
Registro</h4>
          </div>
          <div class="card-body">
            <form id="formulario vehiculo">

              <input type="hidden" id="id" name="id">
   </div>

   <h1 class="texto-negro"> Ingrese los datos del Vehiculo </h1>

   <br>

    <div class="container mt-5 ">
    <div class="row justify-content-center">
      <div class="col-md-5 col-lg-3 test_form">
        <div class="card shadow">
          <div class="card-header ">
            <h4 class="texto-formulario mb-0 text-center texto-blanco"
>Formulario de Registro</h4>
          </div>
          <div class="card-body">

            <div class="mb-3">
              <label for="precio" class="form-label">precio</label>
              <input type="number" class="form-control" id="precio"
placeholder="100$" required>
```

```
            </div>

            <div class="mb-3">
              <label for="año" class="form-label">año</label>
              <input type="number" class="form-control" id="año"
required>
            </div>

            <div class="mb-3">
              <label for="modelo" class="form-label">modelo</label>
              <input type="text" class="form-control" id="modelo"
required>
            </div>
              <div class="mb-3">
              <label for="id_empleado"
class="form-label">id_Empleado</label>
              <input type="text" class="form-control" id="id_empleado"
required>
            </div>
             <div class="mb-3">
              <label for="Marca" class="form-label">Marca</label>
              <input type="text" class="form-control" id="Marca" required>
            </div>



            <button type="submit" class="btn btn-primary w-100
hover-lift">Enviar</button>
            <p id="mensaje"></p>
          </form>
        </div>
```

```
      </div>
    </div>
  </div>
</div>

</div>



<script>
const formulario = document.getElementById("formulario vehiculo");
const mensaje = document.getElementById("mensaje");

const parametros = new URLSearchParams(window.location.search);
const id = parametros.get("id");

if (!id || !/^[0-9]+$/.test(id)) {
    mensaje.className = "alert alert-danger";
    mensaje.textContent = "El ID del vehiculo  no es válido.";
    formulario.style.display = "none";
} else {
    fetch("/api/Vehiculos")
        .then(function (respuesta) {
            if (!respuesta.ok) {
                throw new Error("No se pudo consultar la vehiculo .");
            }
            return respuesta.json();
        })
        .then(function (vehiculos) {
            // Buscamos el elemento actual por ID
            const vehiculo = vehiculos.find(function (elemento) {
```

```
                return String(elemento.id) === id;
            });

            // CORREGIDO: Antes evaluabas '!paquete' cuando tu variable se llamaba 'Adheridos'
            if (!vehiculo) {
                throw new Error("No se encontró el Vehiculo .");
            }

            // CORREGIDO: Mapeo manual directo a los IDs reales de tus inputs en el HTML
            document.getElementById("precio").value = vehiculo.precio || "";
            document.getElementById("año").value = vehiculo.anio || "";
            document.getElementById("modelo").value = vehiculo.modelo || "";
            document.getElementById("id_empleado").value = vehiculo.empleado || "";
            document.getElementById("Marca").value = vehiculo.marca || "";
        })
        .catch(function (error) {
            mensaje.className = "alert alert-danger";
            mensaje.textContent = error.message;
            formulario.style.display = "none";
        });
}

formulario.addEventListener("submit", function (evento) {
    evento.preventDefault();
```

```
        // CORREGIDO: Los nombres de las propiedades deben coincidir con lo que espera tu API de edición
        const vehiculo = {
            id_vehiculo: id,
            precio: document.getElementById("precio").value,
            año: document.getElementById("año").value,
            modelo : document.getElementById("modelo").value,
            id_empleado : document.getElementById("id_empleado").value,
            marca : document.getElementById("Marca").value
        };

        fetch("/api/Vehiculos/editar", {
            method: "POST",
            headers: {
                "Content-Type": "application/x-www-form-urlencoded"
            },
            body: new URLSearchParams(vehiculo).toString()
        })
        .then(function (respuesta) {
            return respuesta.json().then(function (datos) {
                if (!respuesta.ok) {
                    throw new Error(datos.error || "No se pudo editar.");
                }
                return datos;
            });
        })
        .then(function (datos) {
            mensaje.className = "alert alert-success";
            mensaje.textContent = datos.mensaje || "Editado correctamente.";
```

```
            setTimeout(function () {
                window.location.href = "/Mostrar Vehiculo.html";
            }, 1000);
        })
        .catch(function (error) {
            mensaje.className = "alert alert-danger";
            mensaje.textContent = error.message;
        });
    });
</script>


</body>
</html>
```

y esta es un como fue quedando el proyecto hasta llegar a la culminacion del mismo



25/10

Una de mis ultimas tareas fueron realizar los triggers de mis respectivos campos

Codigo :

Devolver Stock si se desarma o elimina un Producto de un Paquete

```
DELIMITER //

CREATE TRIGGER trg_devolver_stock_productos
AFTER DELETE ON Productos_Paquetes
FOR EACH ROW
BEGIN
    UPDATE Productos
    SET stock = stock + OLD.cantidad
    WHERE ID_Producto = OLD.ID_Producto;
END //

DELIMITER ;
```

Evitar Stocks Negativos y Precios Inválidos

```
DELIMITER //

CREATE TRIGGER trg_productos_antes_insertar
BEFORE INSERT ON Productos
FOR EACH ROW
BEGIN

    IF NEW.stock < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El stock inicial del producto no
puede ser negativo.';
    END IF;

    IF NEW.precio <= 0 THEN
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Error: El precio del producto debe ser
mayor a cero.';
    END IF;
END //

DELIMITER ;
```

Control de Stock y Precios al Modificar

```
DELIMITER //

CREATE TRIGGER trg_productos_antes_actualizar
BEFORE UPDATE ON Productos
FOR EACH ROW
BEGIN
    IF NEW.stock < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: La actualización dejaría el stock en
negativo.';
    END IF;
    IF NEW.precio <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El precio modificado debe ser
mayor a cero.';
    END IF;
END //

DELIMITER ;
```

Validar Peso, Volumen y Fecha

```
DELIMITER //
```

```
CREATE TRIGGER trg_paquetes_antes_insertar
BEFORE INSERT ON Paquetes
FOR EACH ROW
BEGIN
   IF NEW.peso <= 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Error: El peso del paquete debe ser mayor
a cero.';
   END IF;

   IF NEW.volumen <= 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Error: El volumen del paquete debe ser
mayor a cero.';
   END IF;

   IF NEW.fecha_ensamble > CURDATE() THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Error: La fecha de ensamble no puede ser
una fecha futura.';
   END IF;
END //

DELIMITER ;
```

Control de Datos al Modificar

```
DELIMITER //

CREATE TRIGGER trg_paquetes_antes_actualizar
BEFORE UPDATE ON Paquetes
FOR EACH ROW
BEGIN
   IF NEW.peso <= 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El peso modificado debe ser mayor
a cero.';
    END IF;

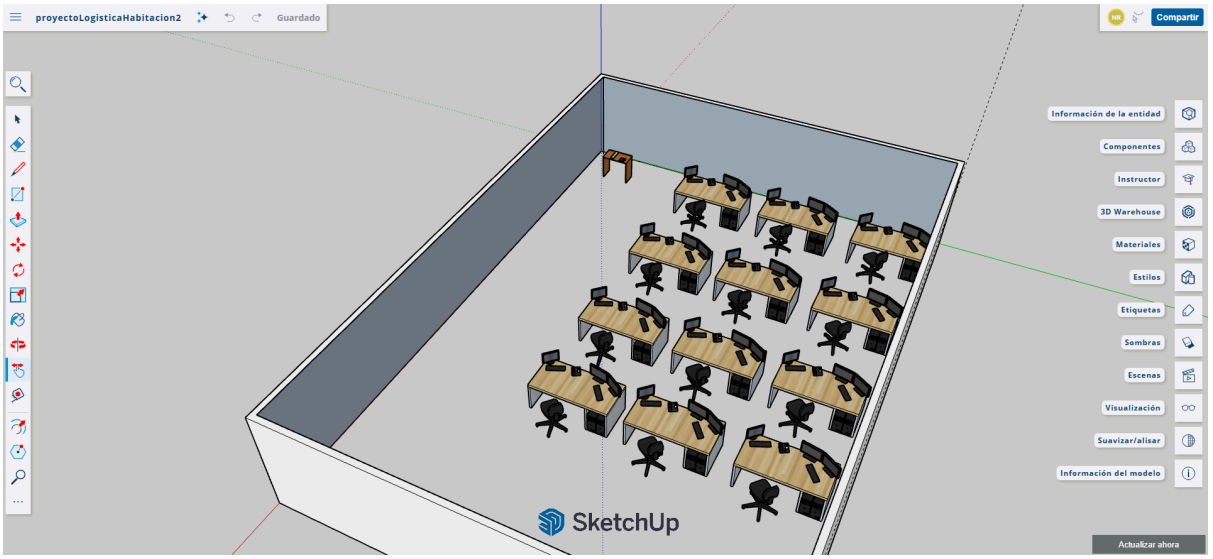
    IF NEW.volumen <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El volumen modificado debe ser
mayor a cero.';
    END IF;

    IF NEW.fecha_ensamble > CURDATE() THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: No se puede asignar una fecha de
ensamble futura.';
    END IF;
END //

DELIMITER ;
```

27/10

Se continuó con el modelado tridimensional del proyecto de infraestructura logística utilizando la herramienta SketchUp. Durante esta jornada, el desarrollo se centró en el diseño y la renderización a escala de la segunda habitación de las instalaciones, optimizando la distribución espacial para el almacenamiento seguro y la correcta circulación de la paquetería dentro del centro de distribución.



31/10

Se ejecutaron los ajustes finales y las modificaciones de depuración sobre las interfaces maquetadas en HTML. Con estas tareas de optimización de las plantillas visuales, se dio por concluida formalmente la etapa de integración y establecimiento de conexiones entre la capa de presentación y la lógica de control desarrollada en el backend de Java.

2/11

Se llevó a cabo una sesión de desarrollo conjunto con el compañero Benjamin Saragusti con el propósito de resolver las últimas tareas pendientes en la capa de código. Durante esta jornada, se revisaron las funciones de consistencia y se realizaron las pruebas unitarias finales, alcanzando el cierre técnico de la programación de los módulos asignados y asegurando la estabilidad integral del software.

6/11

En la fecha actual, se procedió a la elaboración y redacción técnica del Manual de Explotación del sistema informático logístico, contando con la colaboración del compañero Benjamin Saragusti. Este documento formal describe detalladamente los requisitos de infraestructura, la arquitectura del sistema, el proceso de despliegue y los pasos secuenciales necesarios para realizar la correcta instalación y puesta en marcha del programa en el entorno operativo.

Manual de Explotación

El proceso de despliegue comienza con la preparación del entorno de la base de datos. Para ello, lo primero que se debe hacer es ingresar al repositorio oficial del proyecto en GitHub y navegar directamente hacia la sección o carpeta denominada **SQL**. Una vez allí, se debe hacer clic sobre el botón verde **"Code"** para desplegar las opciones y seleccionar la descarga del proyecto completo en formato **.zip**. Tras completar la descarga y guardar el archivo en el equipo, es necesario abrir el Panel de Control de la herramienta **XAMPP** y proceder a iniciar los servicios esenciales haciendo clic en el botón **"Start"** de forma individual para los módulos de **Apache**, **MySQL** y **FileZilla**.

